



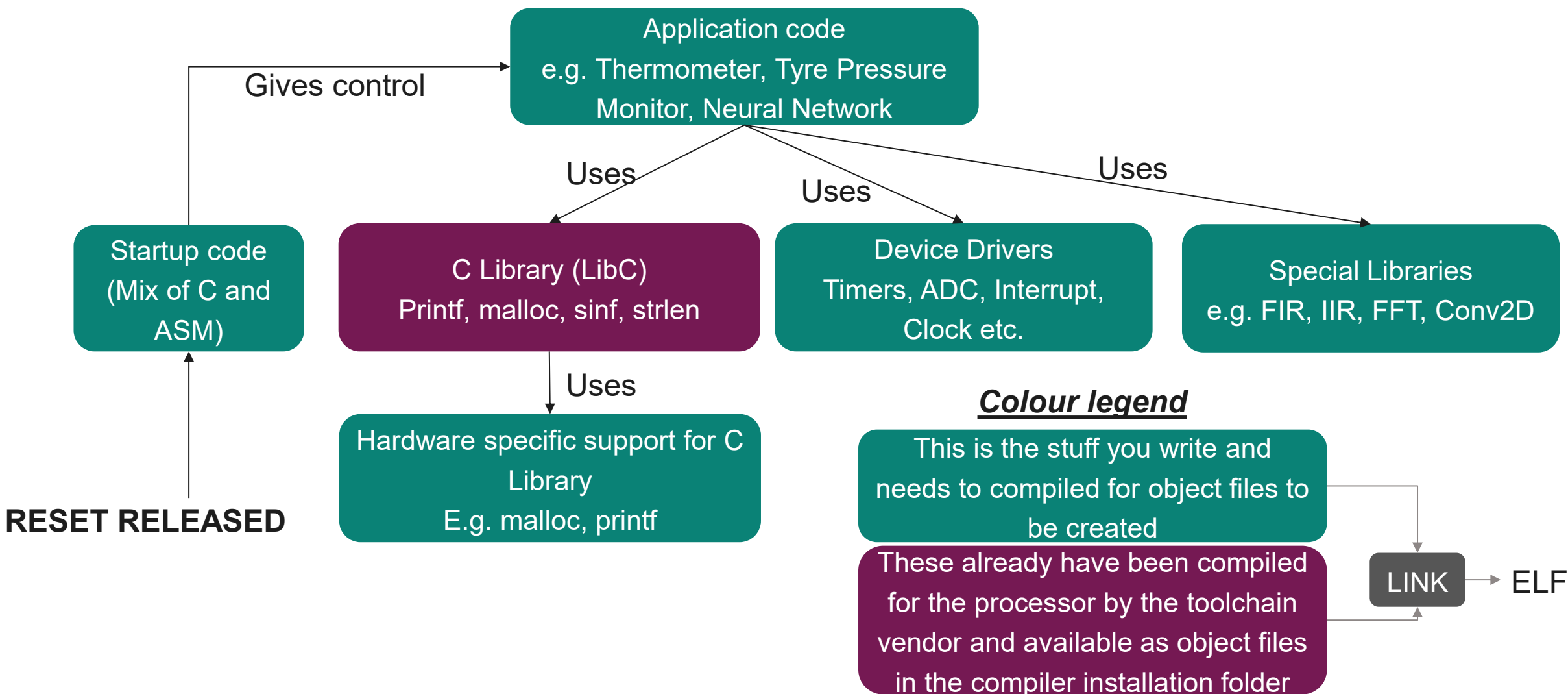
MAKE FILES

Boya Vinay Kumar

18 Oct 2025



A typical baremetal source code tree



Makefile - Motivation

- Build automation infrastructure to manage large projects
- With Makefiles, you can automate
 - Compilation of a large number of source code files, Linking of objects and generation of ELF
 - Other ancillary tasks such as flashing and many more (really!)
- You define dependencies and implicitly the order in which compilation and linking may be done
- You define the rules/commands for each task (rule for compilation, rule for linking, rule for sending SMS...)
- Makefile will start with the top level target that you have defined, understand dependencies among various files and execute the rules set by you!

Dependencies and rules

- ADD.o is an object file
- It is created from ADD.c
- Therefore ADD.o is dependent on ADD.c
- This dependency is expressed in the Makefile as:

ADD.o : ADD.c **DEPENDENCY**

- So a construct that you will regularly see is in a Makefile:

File.o : File.c

arm-none-eabi-gcc <Compile Options> -c File.c -o File.o **RULE**

Application.Elf : File.o Main.o

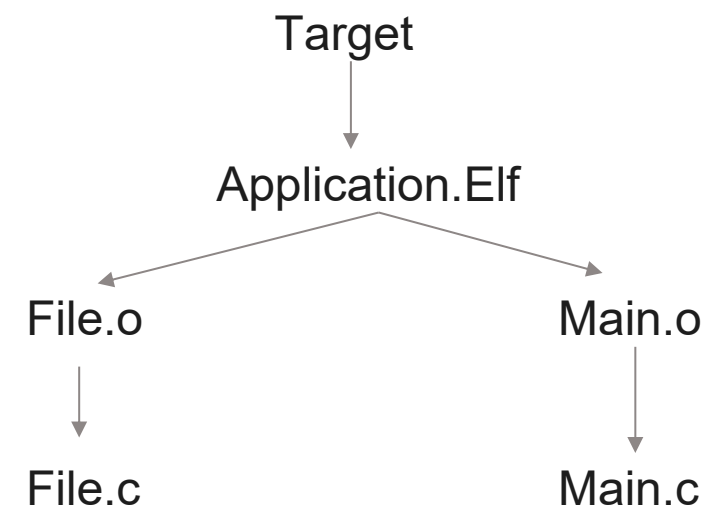
arm-none-eabi-gcc <Link Options> File.o Main.o -o Application.ELF

Target : Application.Elf

How to invoke a Makefile?

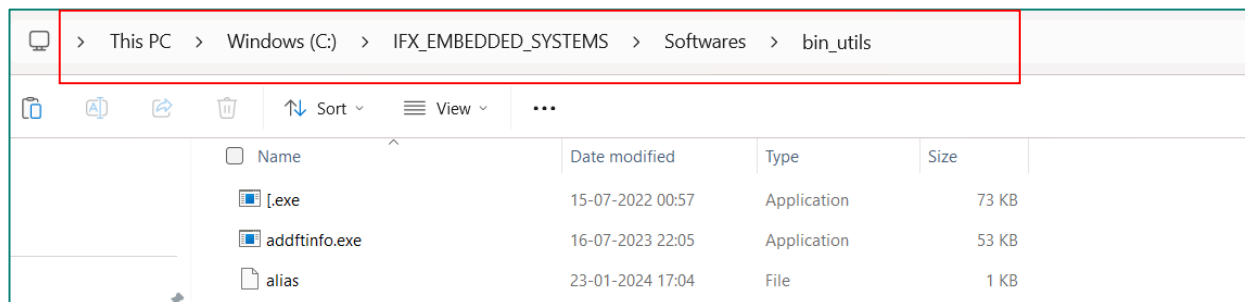
C:\windows_prompt>make Target

How does Makefile establish dependency?



Make -> bin utils installation

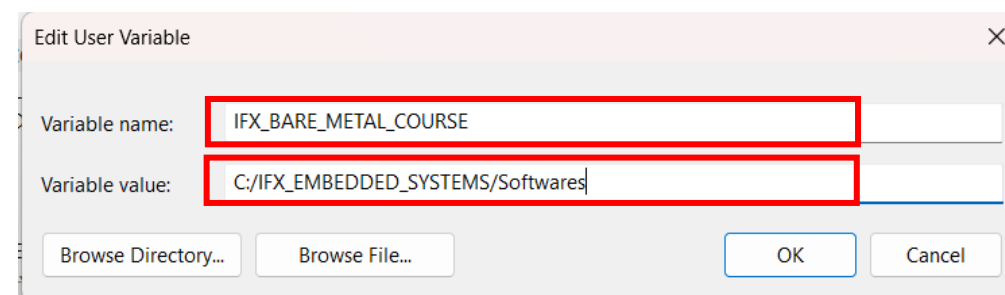
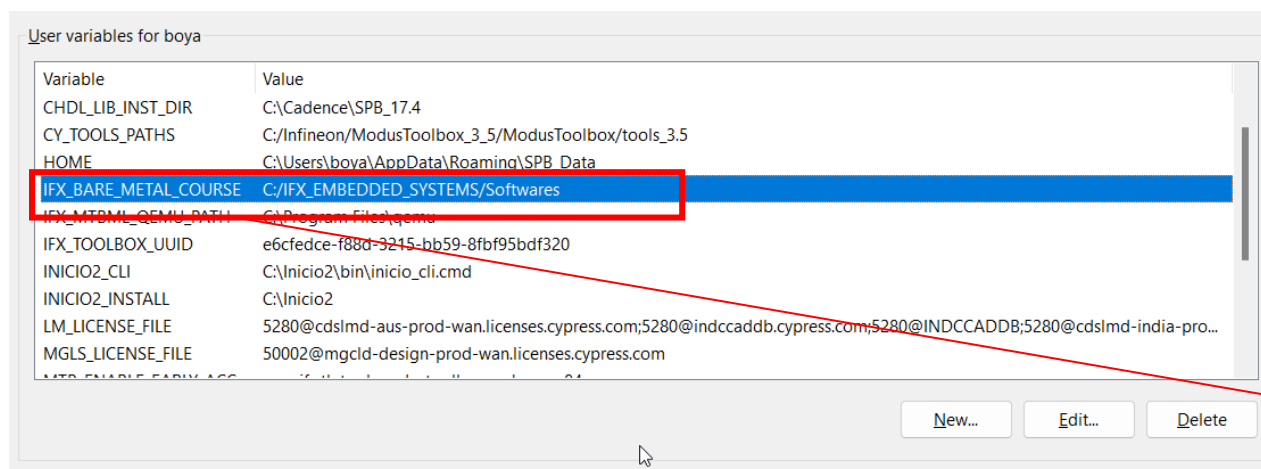
1. Copy and unzip the "**bin_utils.zip**" to the **C:\IFX_EMBEDDED_SYSTEMS\Softwares** folder



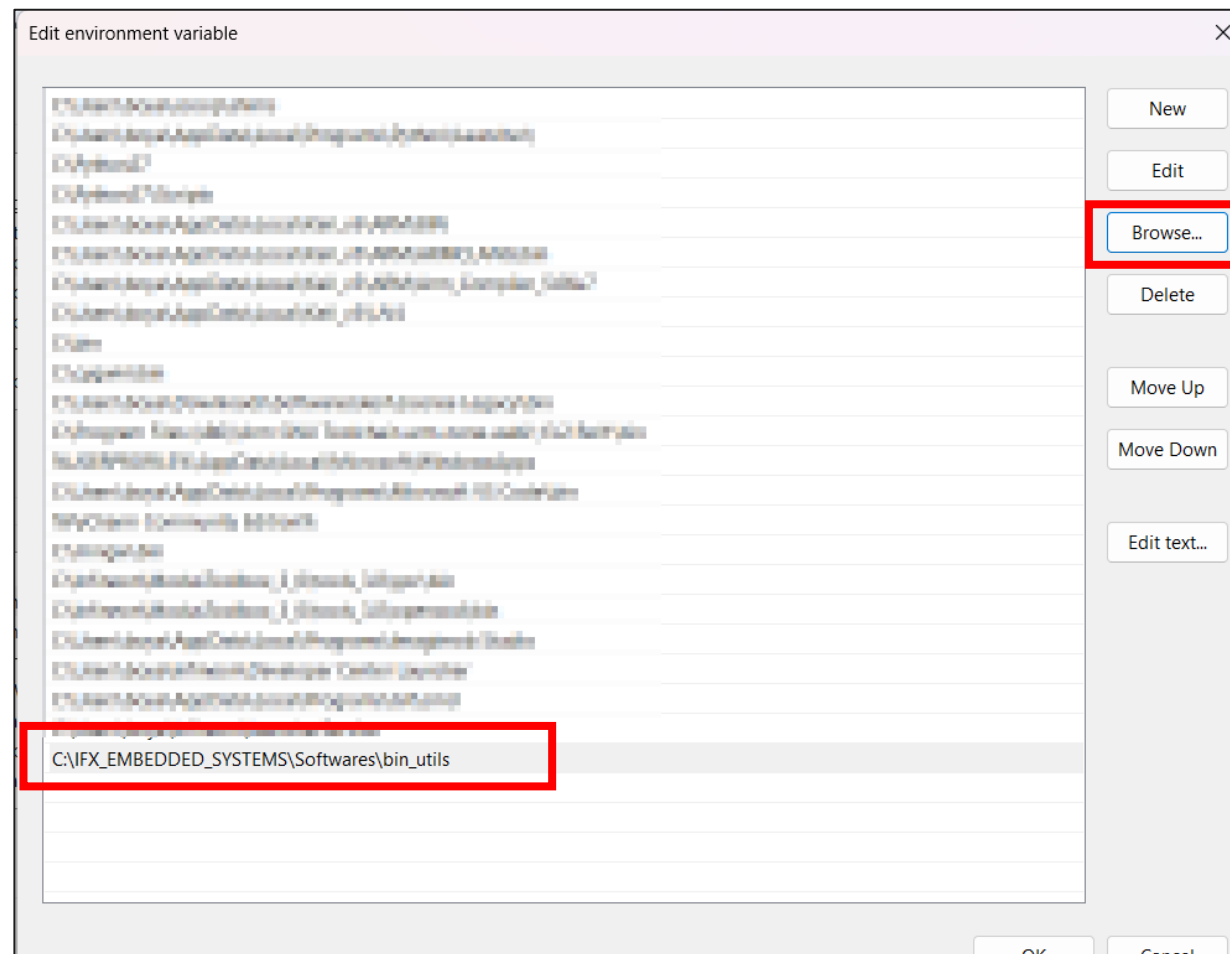
2. Create Environment variable for your account with :-

Variable name= **IFX_BARE_METAL_COURSE**

Path = **C:/IFX_EMBEDDED_SYSTEMS/Softwares** {Unix forward slash}



IFX_BARE_METAL_COURSE



Hierarchical Makefile

